

Introduction to Word Embeddings

Word Representation

Word representation

$V = [a, aaron, \dots, zulu, <UNK>]$ $|V| = 10,000$

1-hot representation

Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$	$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}$

Handwritten notes: Blue arrows point from the words to their respective vectors. A blue arrow points from the 'Apple' vector to the 'Orange' vector. A blue arrow points from the 'Man' vector to the 'Woman' vector. A blue arrow points from the 'King' vector to the 'Queen' vector. A blue arrow points from the 'Apple' vector to the 'Orange' vector. A blue arrow points from the 'Apple' vector to the 'Orange' vector. A blue arrow points from the 'Apple' vector to the 'Orange' vector.

I want a glass of orange juice.
I want a glass of apple _____.

much more similar than king and orange or queen and orange.

Andrew Ng

Word Representation in NLP

- This week focuses on applying **RNNs, GRUs, and LSTMs** to **Natural Language Processing (NLP)**.
- A key concept is **word embeddings**, which provide a better way to represent words than one-hot vectors.
- Word embeddings enable models to:
 - Capture **semantic relationships** between words (e.g., *man : woman = king : queen*).
 - Build effective NLP applications even with **relatively small labeled datasets**.
- The course also introduces **debiasing word embeddings** to reduce unwanted biases (e.g., gender or ethnicity).

One-Hot Word Representation

- Previously, words were represented using a **one-hot vector** based on a fixed vocabulary.
- Example:
 - If **"man"** is word #5391, its vector contains:
 - **1** at position 5391.
 - **0** everywhere else.
- Every word (e.g., *woman, king, queen, apple, orange*) has its own unique one-hot vector.

Limitations of One-Hot Encoding

- One-hot vectors treat **every word as completely independent**.
- They **do not capture semantic similarity** between related words.
- Example:
 - A language model may learn that:
| "I want a glass of orange juice."
 - However, it cannot easily infer that:

| "I want a glass of apple juice."

- because **"apple"** and **"orange"** appear no more related than **"king"** and **"orange."**

Why One-Hot Encoding Fails

- For any two different one-hot vectors:
 - Their **dot product is always 0**.
 - Their **Euclidean distance is the same**.
- As a result, the model cannot recognize that:
 - **apple** is much more similar to **orange**
 - than **apple** is to **king** or **queen**.
- This makes it difficult for models to **generalize knowledge across semantically related words**.

Motivation for Word Embeddings

- Word embeddings solve the limitations of one-hot encoding by:
 - Representing semantically similar words with **similar vectors**.
 - Enabling models to generalize across related words.
 - Capturing meaningful linguistic relationships and analogies automatically.

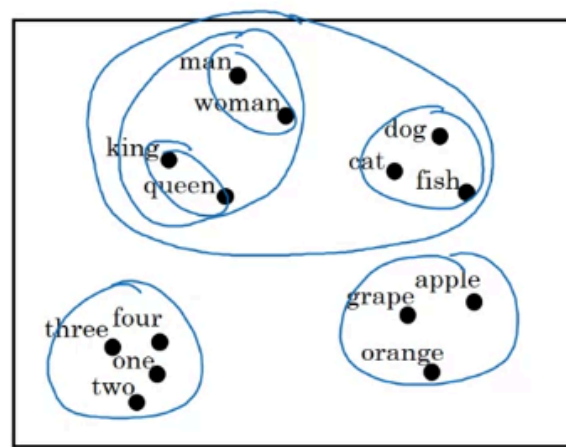
Featurized representation: word embedding

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	<u>0.93</u>	<u>0.95</u>	-0.01	0.00
Age	0.03	0.02	0.7	0.69	0.03	-0.02
Food	0.04	0.01	0.02	0.01	0.95	0.97
size	⋮	⋮				
cost	⋮	⋮				
alive	⋮	⋮				
verb	⋮	⋮				

I want a glass of orange juice.
I want a glass of apple juice.
Andrew Ng

How about each number of a vector represent for a feature? in that case: the apple vector is near to the orange vector

Visualizing word embeddings



[van der Maaten and Hinton., 2008. Visualizing data using t-SNE]

Andrew Ng

From One-Hot Vectors to Word Embeddings

- Instead of representing each word as a one-hot vector, we learn a **dense feature vector (embedding)** for every word.
- Each embedding is a **high-dimensional vector** (e.g., 300 dimensions) that captures semantic properties.
- Example notation:
 - **e₅₃₉₁** → embedding for *man*.
 - **e₉₈₅₃** → embedding for *woman*.

Intuition Behind Word Embeddings

- A word embedding can be viewed as a collection of features describing a word.
- Example conceptual features:
 - **Gender** (male ↔ female)
 - **Royalty** (ordinary ↔ royal)
 - **Age**
 - **Food or not**
 - Other possible features:
 - Size
 - Cost
 - Living/non-living
 - Part of speech (noun, verb, etc.)
- **Note:** These are only intuitive examples. In practice, the learned dimensions **do not have explicit human-interpretable meanings**.

Advantages of Word Embeddings

- Semantically similar words have **similar embedding vectors**.
- Example:
 - **apple** and **orange** have many shared features, so their embeddings are close.
 - **king** and **orange** are much less similar.
- This enables models to **generalize** from one word to another.

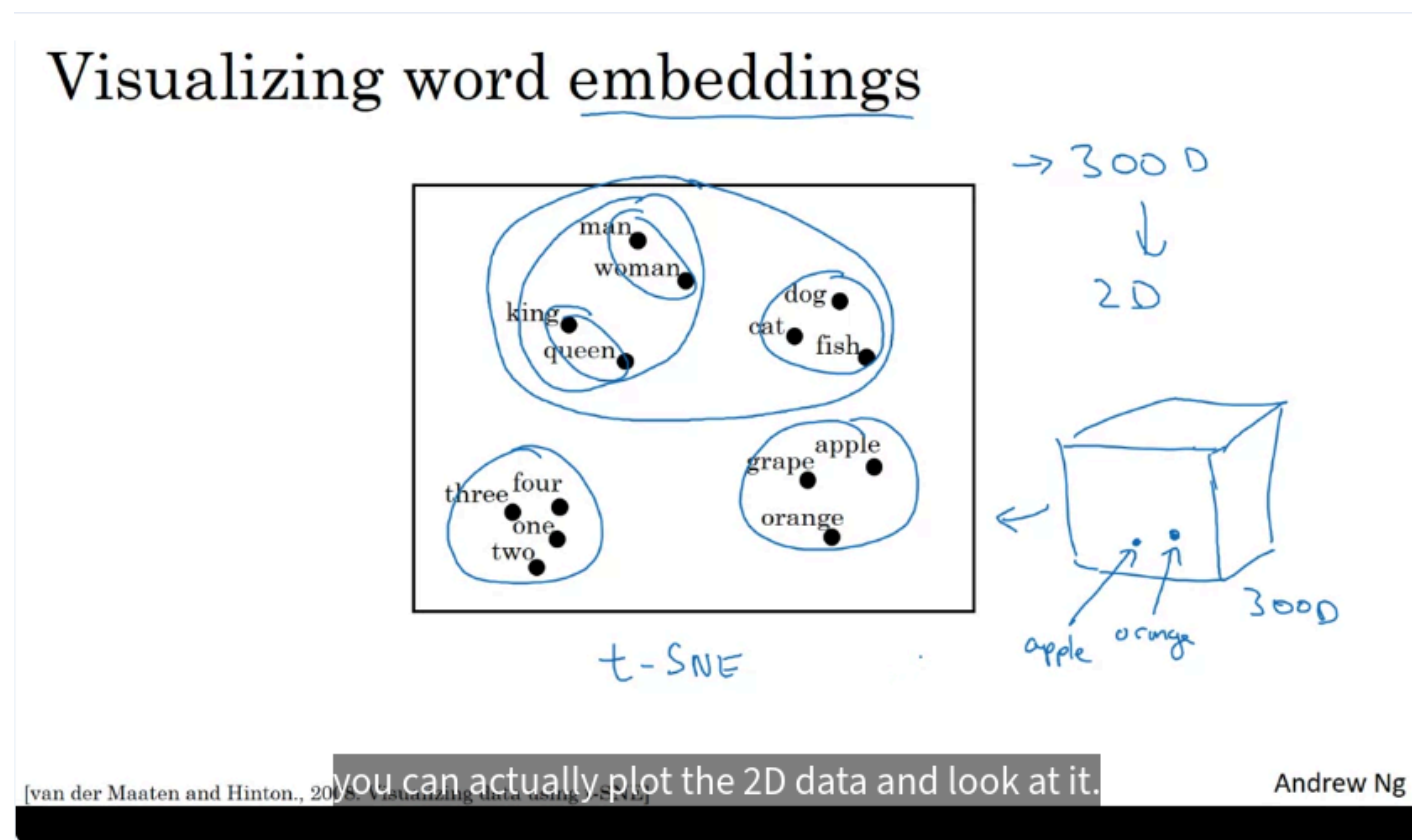
- If the model learns "**orange juice**", it can more easily infer "**apple juice.**"

Learning Word Embeddings

- The goal is to automatically learn a **high-dimensional embedding** (typically around 300 dimensions) for every word.
- Unlike manually designed features, the learned dimensions are:
 - Distributed.
 - Difficult to interpret individually.
 - Effective at capturing semantic relationships.

Visualizing Word Embeddings

- Since embeddings are high-dimensional, they are often projected into **2D** for visualization.
- A common dimensionality reduction algorithm is **t-SNE**.
- After projection, semantically related words naturally cluster together:
 - **man, woman**
 - **king, queen**
 - Animals
 - Fruits
 - Numbers
 - Other related concepts



Why They Are Called "Embeddings"

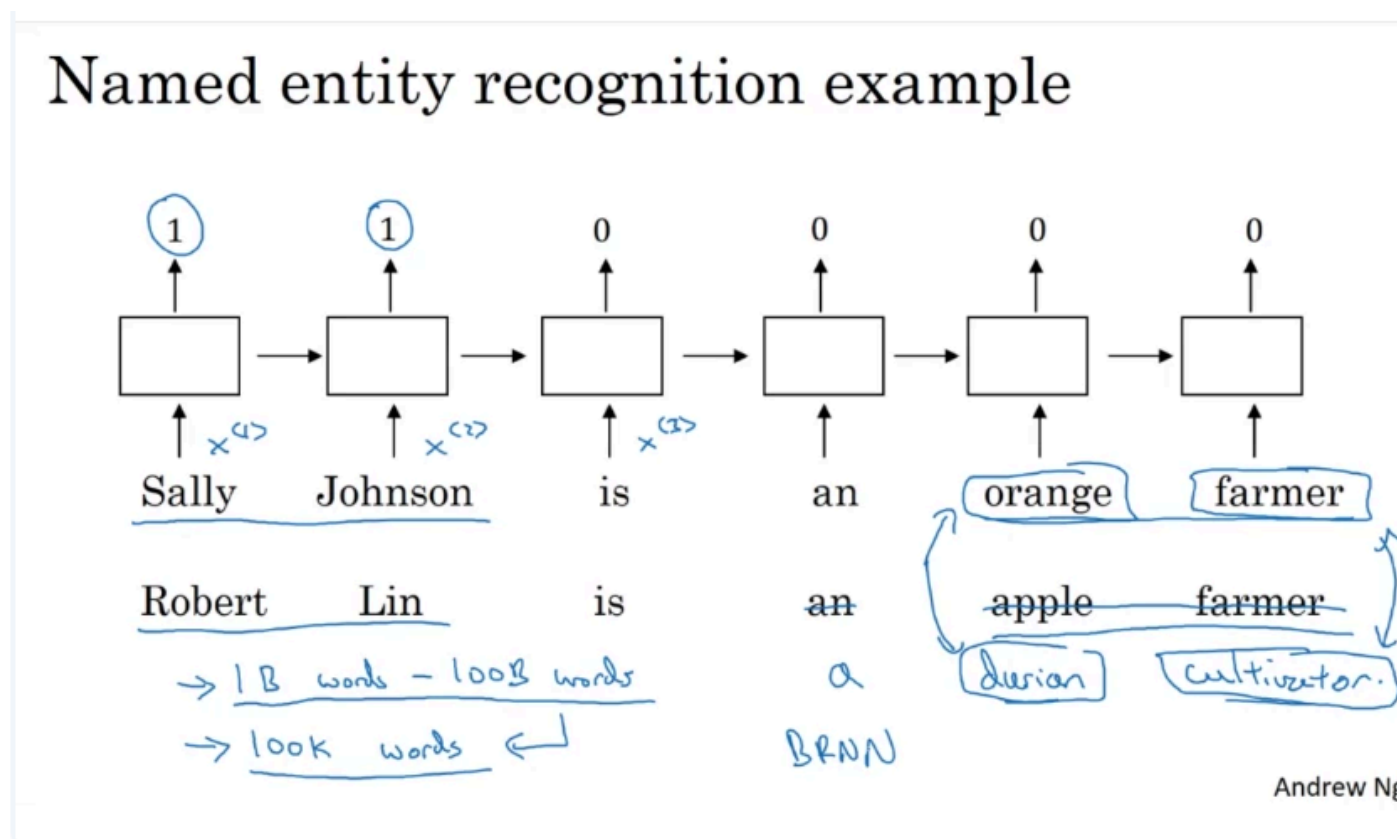
- Each word is mapped (**embedded**) into a point in a **high-dimensional vector space**.
- Similar words occupy nearby locations in this space.
- Visualization methods like **t-SNE** project this space into 2D so humans can inspect the learned structure.

Key Takeaways

- Word embeddings replace sparse one-hot vectors with **dense semantic representations**.
- Similar words receive **similar vectors**, allowing better knowledge transfer and generalization.

- Embeddings are a foundational idea in **Natural Language Processing (NLP)** and significantly improve the performance of many NLP models.

Using Word Embeddings



Transfer learning and word embeddings

1. Learn word embeddings from large text corpus. (1-100B words)

(Or download pre-trained embedding online.)

2. Transfer embedding to new task with smaller training set.
(say, 100k words)

→ 10,000 → 300

3. Optional: Continue to finetune the word embeddings with new data.

Andrew Ng

Using Word Embeddings in NLP

- Word embeddings can replace one-hot vectors as inputs to NLP models.
- They provide semantic information, allowing models to generalize beyond exact words seen during training.
- Example:
 - Training: "**Sally Johnson** is an **orange** farmer."
 - Test: "**Robert Lin** is an **apple** farmer."
 - Since **apple** and **orange** have similar embeddings, the model can still recognize **Robert Lin** as a person's name.

Better Generalization to Unseen Words

- Word embeddings help even when the test sentence contains **rare or unseen words**.
- Example:
 - **"Robert Lin is a durian cultivator."**
- Even if **durian** and **cultivator** never appear in the labeled training set:
 - Embeddings may already capture that:
 - **durian** $\approx$ **orange (fruit)**
 - **cultivator** $\approx$ **farmer**
- This allows the model to correctly infer that **Robert Lin** is likely a person's name.

Transfer Learning with Word Embeddings

Step 1: Pretrain Word Embeddings

- Learn embeddings from a **very large unlabeled text corpus** (millions or billions of words).
- Alternatively, download **pretrained embeddings**.

Step 2: Apply to a New NLP Task

- Use the pretrained embeddings as word representations.
- Train on a **much smaller labeled dataset** for the target task.
- This transfers semantic knowledge learned from massive unlabeled data.

Benefits of Transfer Learning

- Massive unlabeled text teaches relationships such as:
 - orange $\leftrightarrow$ apple $\leftrightarrow$ durian (fruits)
 - farmer $\leftrightarrow$ cultivator
- Small labeled datasets no longer need to learn these relationships from scratch.
- This significantly improves performance when labeled data is limited.

Dense Embeddings vs. One-Hot Vectors

One-Hot Vector	Word Embedding
Very high dimensional (e.g., 10,000)	Low dimensional (e.g., 300)
Sparse	Dense
No semantic similarity	Captures semantic similarity
Poor generalization	Strong generalization

Fine-Tuning Word Embeddings

- After transferring embeddings to a new task, there are two options:
 - **Freeze** the embeddings (most common when labeled data is small).
 - **Fine-tune** the embeddings if the target dataset is sufficiently large.
- Fine-tuning allows embeddings to adapt to the specific task.

When Word Embeddings Are Most Useful

Word embeddings provide the greatest benefit when:

- There is a **huge amount of unlabeled text** available.
- The target NLP task has **limited labeled data**.

They have been highly effective for:

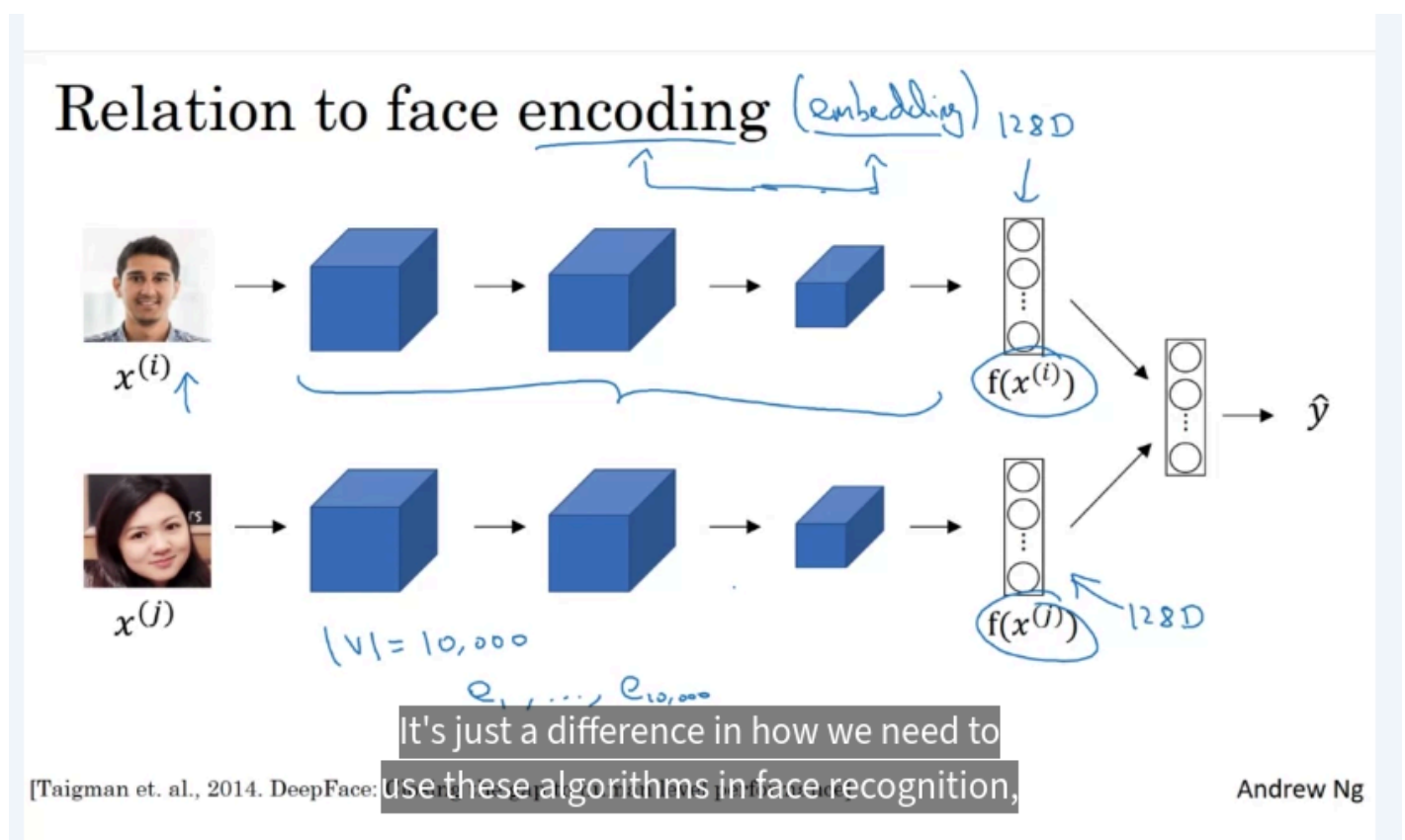
- Named Entity Recognition (NER)
- Text Summarization
- Coreference Resolution
- Parsing

They are **less beneficial** for:

- Language Modeling
- Machine Translation

when these tasks already have **very large labeled training datasets**.

Relationship to Face Encodings



Word embeddings are conceptually similar to **face encodings** learned by Siamese Networks.

Similarity

- Both represent inputs as dense vectors.
- Similar inputs are mapped to nearby points in the embedding space.

Key Difference

Face Encoding	Word Embedding
Computes an encoding for any new face image	Learns a fixed embedding for each word in a fixed vocabulary
Can encode unseen images	Unknown words are usually mapped to a special UNK token

Key Takeaways

- Word embeddings enable **transfer learning** in NLP.
- They allow models to generalize to **similar or previously unseen words**.
- Pretraining on massive unlabeled text provides valuable semantic knowledge that can be transferred to downstream tasks with limited labeled data.
- Compared to one-hot vectors, embeddings are **dense, lower-dimensional, and semantically meaningful**, leading to better performance on many NLP tasks.

Properties of Word Embeddings

Analogy

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	Apple (456)	Orange (6257)
Gender	-1	1	-0.95	0.97	0.00	0.01
Royal	0.01	0.02	0.93	0.95	-0.01	0.00
Age	0.03	0.02	0.70	0.69	0.03	-0.02
Food	0.09	0.01	0.02	0.01	0.95	0.97

$e_{man} - e_{woman} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$
 $e_{king} - e_{queen} \approx \begin{bmatrix} -2 \\ 0 \\ 0 \\ 0 \end{bmatrix}$
 $e_{man} - e_{woman} \approx e_{king} - e_{?}$

$e_{man} = \begin{bmatrix} -1 \\ 0.01 \\ 0.03 \\ 0.09 \end{bmatrix}$
 $e_{woman} = \begin{bmatrix} 1 \\ 0.02 \\ 0.02 \\ 0.01 \end{bmatrix}$
 $e_{king} = \begin{bmatrix} -0.95 \\ 0.93 \\ 0.70 \\ 0.02 \end{bmatrix}$
 $e_{queen} = \begin{bmatrix} 0.97 \\ 0.95 \\ 0.69 \\ 0.01 \end{bmatrix}$

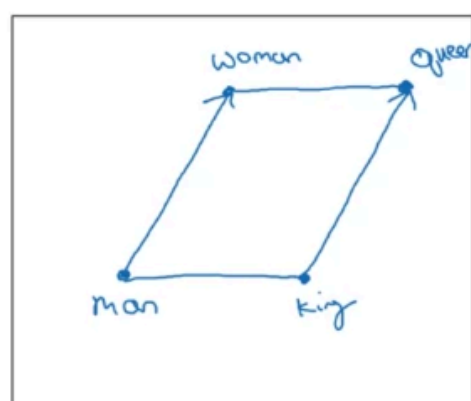
$e_{man} - e_{woman} \approx e_{king} - e_{?}$
 $e_{man} - e_{woman} \approx e_{king} - e_{queen}$

[Mikolov et. al., 2013, Linguistic regularities in continuous space word representations]

Andrew Ng

This is the idea how to learn the representation where these vectors: $e_{man} - e_{woman} \approx e_{king} - e_{queen}$

Analogy using word vectors



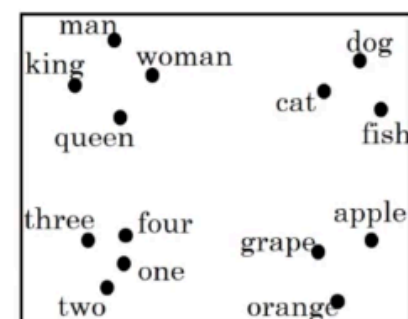
300D

Find word w : $\arg \max_w$

$$e_{man} - e_{woman} \approx e_{king} - e_w$$

$$\text{Sim}(e_w, e_{king} - e_{man} + e_{woman})$$

30 - 75%



t-SNE

Andrew Ng

how to find the word w ? We find the w by find the below: $\arg \max_w (\text{similarity}(e_w, e_{king} - e_{man} + e_{woman}))$

Because the **t-SNE** (t-Distributed Stochastic Neighbor Embedding) is **non-linear**, these geometric relationships are generally **not preserved** in the 2D visualization. So we shouldn't count on that much

Word Embeddings Can Solve Analogies

- One remarkable property of word embeddings is their ability to perform **analogy reasoning**.
- Example:
 - man : woman = king : queen**
- This demonstrates that embeddings capture meaningful semantic relationships between words.

Vector Arithmetic Behind Analogies

- Each word is represented by an embedding vector.
- Relationships between words can be represented by **vector differences**.
- Example:

- **$e(\text{man}) - e(\text{woman}) \approx e(\text{king}) - e(\text{queen})$**
- The difference mainly captures the concept of **gender**.
- Similar relationships exist for many other word pairs.

Solving Analogies with Embeddings

To answer:

man : woman = king : ?

the algorithm computes:

$$e(\text{king}) - e(\text{man}) + e(\text{woman})$$

Then it searches for the word whose embedding is **most similar** to this resulting vector.

If the embeddings are well learned, the closest word is **queen**.

Analogy Search Formula

The algorithm finds the word **w** that maximizes:

$$\text{Similarity}(e(w), e(\text{king}) - e(\text{man}) + e(\text{woman}))$$

- Similarity is usually measured using **cosine similarity**.
- This method often achieves **30%–75% accuracy** on standard analogy benchmarks (requiring the exact correct word).

Cosine Similarity

The most commonly used similarity metric is:

$$\text{cosine}(u, v) = \frac{u^T v}{\|u\| \|v\|}$$

Interpretation

- **1** → vectors point in the same direction (very similar).
- **0** → vectors are orthogonal (unrelated).
- **-1** → vectors point in opposite directions.

Why Cosine Similarity?

- Measures the **direction** of vectors rather than their magnitude.
- More commonly used than Euclidean distance for comparing word embeddings.

t-SNE Visualization vs. Original Embedding Space

- Word embeddings are usually learned in a **high-dimensional space** (e.g., 300 dimensions).
- Algorithms like **t-SNE** project them into **2D** for visualization.
- Important:
 - The vector arithmetic used for analogies is valid in the **original embedding space**.
 - Because t-SNE is **nonlinear**, these geometric relationships are generally **not preserved** in the 2D visualization.

Examples of Learned Analogies

Word embeddings can automatically discover many semantic relationships, including:

Relationship	Example
Gender	man : woman = boy : girl
Royal title	king : queen

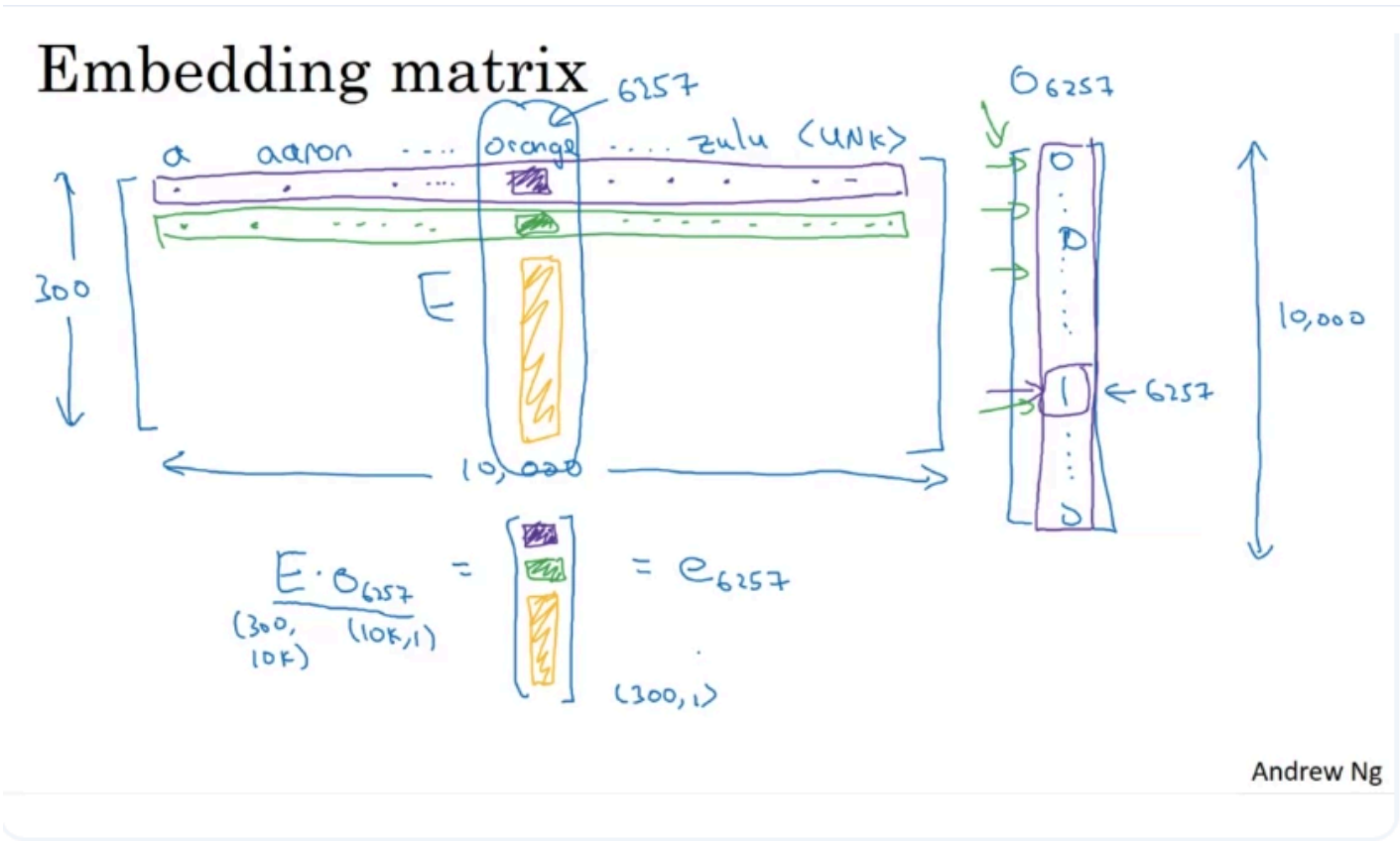
Relationship	Example
Capital → Country	Ottawa : Canada = Nairobi : Kenya
Comparative adjective	big : bigger = tall : taller
Currency → Country	yen : Japan = ruble : Russia

These relationships emerge simply from training on **large text corpora**, without manually encoding linguistic rules.

Key Takeaways

- Word embeddings capture semantic relationships through **vector arithmetic**.
- Analogies can be solved by combining and comparing embedding vectors.
- **Cosine similarity** is the standard metric for comparing embeddings.
- High-quality embeddings naturally encode many linguistic patterns, such as:
 - Gender
 - Capital–country
 - Comparative forms
 - Currency–country
- These properties provide strong evidence that embeddings learn meaningful semantic representations.

Embedding Matrix



Learning Word Embeddings

- Instead of directly learning individual word vectors, the algorithm learns an **embedding matrix**.
- This matrix stores the embedding of **every word in the vocabulary**.

Embedding Matrix

Assume:

- Vocabulary size = **10,000 words**
- Embedding dimension = **300**

Then the embedding matrix is: $E \in \mathbb{R}^{300 \times 10000}$

- **Rows (300):** embedding dimensions (features).

- **Columns (10,000):** embedding vectors for each vocabulary word.
 - Each column corresponds to the embedding of one word.
-

One-Hot Vector Representation

Each word is represented by a **one-hot vector**.

Example:

- **Orange** is word **#6257**.

Its one-hot vector: o_{6257}

- Dimension: **$10,000 \times 1$**
 - Contains:
 - **1** at position **6257**
 - **0** everywhere else
-

Retrieving an Embedding

To obtain the embedding of a word: $e_{6257} = E \cdot o_{6257}$

Dimensions:

- (E): **$300 \times 10,000$**
 - ($o_{\{6257\}}$): **$10,000 \times 1$**
 - Result:
 - **300×1** embedding vector
-

Why Matrix Multiplication Works

Since the one-hot vector contains only **one non-zero element**:

- Every multiplication is with **0**, except at the word's index.
- Therefore: $E \cdot o_{6257}$

simply **selects the 6257-th column** of the embedding matrix.

In other words: $E \cdot o_{6257} = e_{6257}$

So multiplying by a one-hot vector is simply a mathematical way of **looking up the embedding vector** for a word.

Key Takeaways

- Word embeddings are stored in a single **embedding matrix** (E).
- Every **column** of (E) represents one word's embedding.
- A one-hot vector acts as an **index** into the embedding matrix.
- Multiplying the embedding matrix by a one-hot vector simply retrieves the corresponding word embedding efficiently.